

Kapitel 3. Aritmetik

Innehållsförteckning

[Normala aritmetiska operatorer](#)

[Mera om division](#)

[Binär aritmetik](#)

[Inkrementerings- och dekrementeringsoperatorerna ++ och --](#)

[Sammansatta tilldelningar](#)

Detta kapitel behandlar olika aritmetiska möjligheter i C++. Med detta avses bl.a. hur man kan räkna med variabler, precedens mellan operatorer och binära operatorer.

Normala aritmetiska operatorer

Man kan exakt som i andra språk använda sig av de normala aritmetiska operatorerna +, -, * och / för att manipulera tal och variabler i C++. Detta fungerar precis som i vilket programmeringsspråk som helst. Några exempel:

```
int Summa = 10 + 20;
int Area = Kant1 * Kant2;
float aPris = TotalSumma / Antal;
float Kapital = Kapital + Kapital * Ranta;
```

Man kan även använda - för att negera värdet på en variabel, precis som i matematiken. Följande tilldelningar är giltiga:

```
int Negativ = -Positiv;
float Koeff = -10.3 * -Parameter;
```

Jämföra variabler

I C++ används som exemplet ovan visar operatoren == för att jämföra variabler med varandra eller med konstanter. Där evalueras uttrycket `Tal == 27` till `true` om `Tal` har värdet 27, i övriga fall får uttrycket värdet `false`. Många andra språk använder endast = för att jämföra värden, men så alltså inte i C++.

För att testa om någonting är olika något annat används operatoren !=. Denna läses *inte lika med*. T.ex. `Index != 10` evalueras till `true` så länge som `Index` är olika 10.

Storlek kan testas med operatorerna <, >, <= och >=. Dessa fungerar precis som sina matematiska motsvarigheter.

Precedens

Med *precedens* avses den ordning enligt vilken komponenterna i ett uttryck evalueras. C++ följer normal matematisk precedens, vilket gör det lätt att konstruera komplicerade matematiska uttryck. Om alla deluttryck är likvärdiga evalueras uttrycken från vänster till höger. Således evalueras

följande uttryck såsom kan kunde anta:

```
float NyttKapital = Kapital + Kapital * Ranta;  
float TotalArea = Kant1 * Kant2 + Pi * Radie * Radie;  
float Procent = Tal / 100 * Koefficient;
```

I det första uttrycket ovan utförs först `Kapital * Ranta` innan det adderas med `Kapital` och sparas i `NyttKapital`. Trots att `+` kom före utfördes ändå `*` först, eftersom den har högre precedens än `+`. `/` och `*` är likvärdiga, likaså `+` och `-`. I det andra fallet är det heller inga problem och summan av de två areorna evalueras sist. Det tredje fallet är dock en aning problematiskt, eftersom både `*` och `/` har samma precedens. Uttrycket evalueras därför från vänster till höger. Om detta *inte* är vad som avses kan precedensen ändras m.h.a. parenteser. Om man vill utföra divisionen först kan uttrycket skrivas om till:

```
float Procent = Tal / (100 * Koefficient);
```

Parenteser kan fritt användas för att ändra precedens, och kan vara bra att använda för att göra ett komplicerat uttryck lättare att läsa. Man kan sålades skriva om det andra uttrycket på följande sätt för att göra det mera läsligt:

```
float TotalArea = (Kant1 * Kant2) + (Pi * Radie * Radie);
```

En komplett precedenstabell med alla operatorer finns i t.ex. kursboken.

Modulus

Med *modulus* avses resten vid heltalsdivision (se [avsnittet Heltalsdivision](#)). Modulus betecknas med `%`. Några exempel på modulus är:

```
10 % 3 == 1  
6 % 2 == 0
```

Det finns vissa praktiska användningsområden för modulus. Ett exempel är då man vill ha t.ex. entalsdelen från ett valfritt tal:

```
int Heltalsdel = Tal % 10;
```

Andra tillämpning finns även, bl.a. då man manipulerar slumptal och vill ha ett tal som ligger inom ett visst intervall.

[Förutgående](#)
Konstanter

[Hem](#)

[Nästa](#)
Mera om division